A Tour of Pollis AI+

Pollis and AI, working together

Pollis with or without AI

AI is off by default, and Pollis works fully without it: every panel, notebook and workflow runs without AI. Pollis AI+ is for those who want AI beside them: to write a new panel, a toolbox, a method or its vignette, or to polish a report.

Part one: The AI Ecosystem

Talk about AI comes with many new words: language models, runners, agents, harnesses, skills, MCP servers, orchestration. This part explains them for readers who have never met them at a technical level: what each one is, how they fit together, and where each one is in Pollis, with examples. Each topic takes two facing pages; read them in order the first time, since each builds on the one before. Page 14 puts them all in one picture, with the words to know.

Menu paths are written **Compose > Coding Agents**, and buttons as they appear, such as **Use**. What you could ask an agent is written

like this.

This tour describes Pollis as it is on 17 October 2026. What is planned but not available yet is said to be not available.

Getting Pollis

Pollis is free and open source, for Windows, macOS and Linux: github.com/saragga/pollis/releases/latest.

Questions and problems: **Help > Report Issue**, or contact@pollis.org.

Contents

The AI Ecosystem

2 Language Models
4 Local Model Runners
6 Coding Agents and Harnesses
8 Agent Skills
10 MCP Servers
12 Orchestration
14 The Whole Picture

© 2026 Antonio Saragga Seabra · Licensed under CC BY-SA 4.0 · Draft, 17 October 2026

Language Models

In the cloud · On your computer

What a language model is, what it does well and badly, and the difference between one in the cloud and one on your computer.

What a language model is

A *large language model*, or *LLM*, is a program trained on a vast amount of text to do one thing: given some text, continue it. It writes one small piece at a time, a *token*, which is a word or part of a word, each one chosen to follow well from everything before it. Repeated thousands of times, this gives answers, explanations, summaries and code. Claude, GPT and Gemini are language models; so are Gemma, Llama, Qwen and Mistral.

Example

Ask a model *"Write Julia code that fits a linear regression of y on x"* and it writes code, because it has read a great deal of code like it. It has not run that code, and does not know whether it works: it has written the most likely continuation of your request.

What follows from how it works

- **It can be fluently wrong.** A confident answer is not a checked one. A model may call a function a Julia package does not have, or cite an article that was never published. This is why Pollis holds AI work to checks it does not write itself: the tests of a new method are the values the article publishes, and a new panel must pass Pollis' validator.
- **It knows the world up to a date**, its *knowledge cutoff*. A package released after it, or a function renamed since, it does not know, unless it is told. Telling it is the job of skills (page 8) and MCP servers (page 10).
- **It reads a limited amount at once**, its *context window*: your request, the files it was shown, the conversation so far. What is not in the window, it does not see. When the window fills up, the program around the model has to drop or summarise the earlier parts.
- **It does not remember you.** Each conversation starts afresh, unless the program around it keeps notes and shows them to the model next time.
- **Its answers vary.** The same request can give a different answer each time.

A model alone cannot act

A model only writes text. It cannot open your files, run Julia or look anything up. Everything beyond writing text is done by a program around it: for a coding agent, its *harness* (page 6).

Cloud models

Most of the strongest models are run by companies in their data centres: Anthropic's Claude, OpenAI's GPT, Google's Gemini and others. You reach them over the internet, with an account.

- **Strong and fast**, with nothing to install but the program that talks to them.
- **They cost money**: a subscription, or a price per token read and written. A long task reads and writes many tokens. This is why every Pollis menu item that starts an agent carries a sparkle: it runs on your agent's account.
- **Your text goes to the provider**: your request, and every file the agent shows the model. What the provider may do with it is in its terms.
- **They need the internet.**

Local models

An *open-weight* model is one whose trained numbers, its *weights*, are published, so anyone can download it and run it on their own computer: Google's Gemma, Meta's Llama, Alibaba's Qwen, Mistral's models and OpenAI's gpt-oss are examples.

- **Free**, once downloaded.
- **Private**: nothing you give it leaves your computer.
- **Offline**: it works on a train or a plane.
- **Smaller**, so weaker on hard tasks, and slower on a laptop than in a data centre. Your computer's memory limits which models it can run.

	Cloud model	Local model
Runs on	the provider's computers	yours
Costs	a subscription or a price per token	nothing, after the download
Your text	is sent to the provider	stays on your computer
Internet	needed	not needed
Best for	hard tasks, where a mistake costs most	small tasks, private data, no budget

In Pollis

Some tasks are small: **Polish with Agent** tidies the wording of a report. Others are hard: **Write a Method** must reproduce an article's tables to the digits printed. A small, fast or local model suits the first; the second is where a strong model pays for itself. **Model Routing** (page 12) lets you make that choice once for each task.

See also Local Model Runners, page 4. Orchestration, page 12.

Local Model Runners

Compose > Local Models > Local Model Runners

The programs that run a language model on your own computer.

Why a runner is needed

A downloaded model is a large file of numbers; on its own, it does nothing. A *runner* is a program that loads the file into memory, computes with it, on the processor or the graphics chip, and *serves* it: it waits at an address on your computer, so that other programs can send it text and get the model's answer back, as they would from a cloud model. The model file is the recording; the runner is the player.

Addresses and APIs

A runner serves its model at an address such as `http://localhost:11434`. `localhost` means *this computer*; `11434` is the *port*, a door number, so that several programs can serve at once. Programs talk to it through an *API*, an agreed way of asking and answering. The APIs of OpenAI and Anthropic have become common languages, and runners speak them. So a coding agent written for a cloud model can be pointed at a local one by changing one address.

The four runners Pollis knows

Runner	What it is	Port
Ollama	A program that downloads and runs models with one command; OpenAI and Anthropic APIs	11434
LM Studio	A desktop app with a model browser; OpenAI and Anthropic APIs; runs MLX models on Apple silicon	1234
llama.cpp	Its `llama-server`; OpenAI and Anthropic APIs	8080
Apple MLX	Apple's framework for Macs with Apple silicon; its `mlx_lm.server` speaks the OpenAI API	8080

Compose > Local Models > Local Model Runners shows which runners are installed, with **Install Runner** for the others, and how each coding agent reaches each runner. Apple MLX is listed only on a Mac.

Getting a model

Hugging Face Models and **Kaggle Models**, the other entries of **Compose > Local Models**, find models to download. A runner can also fetch them itself. With Ollama, for example:

- `ollama pull gemma3` downloads Google's Gemma 3;
- `ollama run gemma3` starts a chat with it in the terminal.

Which size?

A model's size is its number of weights, written in billions: a "7B" model has about seven billion. Bigger is usually better, and needs more memory. A *quantised* model keeps each weight in fewer bits: a smaller file, slightly less accurate. The rule of thumb: the model's file must fit in your computer's memory, with room to spare for the conversation.

Example: a coding agent on a local model

1. Install Ollama: **Install Runner**, on the Local Model Runners page.
2. In **Compose > Coding Agents**, under **Local and Cloud Models**, use your agent's local command, such as `ollama launch claude -- {prompt}` for Claude Code, or `codex --oss {prompt}` for Codex.
3. Start a task, such as **Compose > New Panels and Toolboxes > Create a Panel...**. The agent works exactly as before; only the model that answers it is now on your computer.

The agent is always local; the model may not be

A coding agent always runs on your computer, in a terminal. What is *local* or *cloud* is the model it talks to. "A local agent" usually means an agent on a local model.

Smaller models, simpler tasks

A local model follows long instructions less reliably than a strong cloud model. Give it the light tasks, and check what it produces, as you would any agent's work.

See also Language Models, page 2. Coding Agents and Harnesses, page 6.

Coding Agents and Harnesses

Compose > Coding Agents

A model that can act: what a coding agent is, and the harness that makes it one.

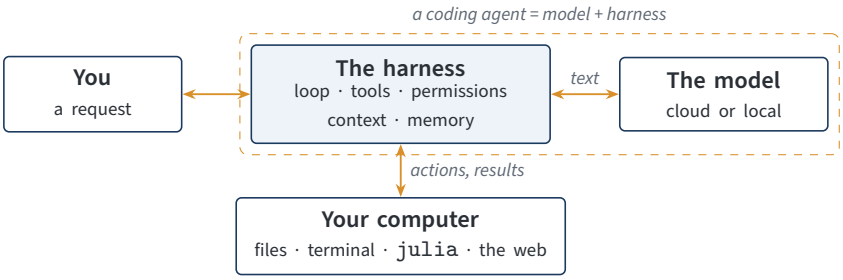
From a chat to an agent

In a chat, you ask, the model answers with code, you copy it into Julia, run it, and paste the error back. A *coding agent* does the copying and the running itself: it reads your files, writes changes, runs commands such as `julia`, reads what they print, and tries again, until the task is done or it needs you.

The harness

The model is the agent's mind; the *harness* is everything around it: a program on your computer, in a terminal, that turns the model's text into actions.

- **The loop.** It sends your request to the model, carries out what the model asks for, sends back the result, and repeats.
- **The tools.** Read a file, edit a file, search, run a command, fetch a web page. The model only asks; the harness does it.
- **The permissions.** Which actions need your yes first, such as running a command or writing outside the project folder.
- **The context.** What goes into the model's window at each step: its instructions, your files, the steps so far, summarised when the window fills up.
- **The memory.** Notes and project instructions kept in files between sessions.



The agents Pollis knows

Compose > Coding Agents lists five agents, each with the command that starts it, and, for most, another that starts it on a local model. Same harness, different model.

Agent	Command	On a local model
Claude Code	`claude {prompt}`	`ollama launch claude -- {prompt}`
Cline	`cline -i {prompt}`	after `cline auth --provider ollama`
Codex	`codex {prompt}`	`codex --oss {prompt}`
GitHub Copilot CLI	`copilot -i {prompt}`	`ollama launch copilot -- -i {prompt}`
OpenCode	`opencode --prompt {prompt}`	`ollama launch opencode -- --prompt {prompt}`

`{prompt}` is where Pollis puts the first message. Click **Use** beside yours; for any other agent, type its command line under **Other Agent**.

Example: Create a Panel, step by step

1. Choose **Compose > New Panels and Toolboxes > Create a Panel...**. Pollis opens a terminal and starts your agent with a first message: *"Read the skill … and follow its instructions to create a new Pollis panel with me."*
2. The agent reads the skill (page 8) and asks you, in one message, what the panel is for, the methods it covers and the Julia packages to use. It proposes a plan and waits for your yes.
3. It writes the panel's TOML file, its wiki pages and its notebook tutorials.
4. It runs Pollis' validator on them, reads the errors, fixes them, and runs it again until it passes.
5. It packages the panel as a toolbox, ready to install or share.

Steps 3 and 4 are the loop: write, check, read the result, correct. A chat cannot do this; an agent can, because its harness can run the validator.

> **An agent works on your files**
>
> It reads and writes files, and runs commands. Read what it proposes before you say yes, and check its results, as you would a colleague's.

See also Agent Skills, page 8. Orchestration, page 12.

Agent Skills

Compose > Agent Skills

Know-how written down, so that an agent can do a task it was never trained for.

Why an agent needs skills

A model knows a great deal in general, and nothing about Pollis' panel format, its registry rules or how its vignettes look: they are too recent, or were never in its training text. Without help, it would guess. A *skill* is the help: the know-how for one task, written down for the agent to read.

What a skill is

A folder with a file named `SKILL.md`: a name, a short description of when to use it, and the instructions. Beside it there can be reference pages and scripts. It is plain text: anyone can read it, and change it, with no retraining of any model. Think of a laboratory protocol: the same protocol makes a new member of the team do the procedure the way the team does it.

How an agent uses one

An agent first reads only the names and descriptions of its skills, which is cheap. When a request matches one, such as

> *Build a toolbox for my course from these panels.*

it reads that skill in full, then follows it. Pollis' agent tasks go straight to the point: their first message names the skill to read.

Scripts that check

The best skills do not only say *check your work*; they bring a program that checks it. The create-panel skill comes with `validatePanel.mjs`, Pollis' validator; the write-method skill runs the method's tests against the published values. Instructions say what to do; the checks show whether it was done.

The Pollis skills

create-panel	Write a new panel on a topic you choose: its TOML file, wiki pages and notebook tutorials; check it with the validator; package it as a toolbox.
package-toolbox	Build a toolbox from the menu entries you pick, in the menus and order you want: for example, one toolbox per course.
write-method	Write the Julia code of a published method, so that it reproduces the article's tables to the digits printed, tested with Pollis' package script.
prepare-method	Bring a method up to the rules of the Pollis registry, such as a docstring for every exported name, without changing what it computes.
write-vignette	Write a method's vignette: a two-page document that sets its results beside the published ones.
references	Cite as Author (year), with all authors written out, each linked to its entry in the references, from the Pollis library, with its DOI.

Examples

💬 *Write a method from this article: these are its tables.*

The write-method skill turns every published value into a test before any code is written, then has the agent write the code until the tests pass.

💬 *Add the references to this report.*

The references skill makes the agent cite from the Pollis library, never from memory, so that no reference is invented.

In Pollis

Compose > Agent Skills lists the skills, downloaded in their latest version when the page opens, each with its **Tasks** and **Example Requests**. **View Skill** opens one; **Run with My Agent** starts your agent on it. **Install Skills**, on the **Compose > Coding Agents** page, installs them for your agent, so that it knows them in any folder, even outside Pollis.

Skills and MCP servers

A skill gives an agent *know-how*: how to do a task. An MCP server, on the next spread, gives it *reach*: data and services it could not get to before.

See also Coding Agents and Harnesses, page 6. MCP Servers, page 10.

MCP Servers

Compose > MCP Servers · Extensions pane > MCP Servers - Recommended

New tools for an agent: data and services it can reach, in a form every agent understands.

What MCP is

A harness comes with general tools: read files, run commands. An *MCP server* adds tools of a particular service: search FRED's series, query a database, read your Zotero library. MCP, the *Model Context Protocol*, is the agreed way an agent and a server talk, so that one server works with every agent that speaks it. Think of a USB plug: the same plug fits every computer.

How a tool is used

Each server lists its tools, with a description of each. The model reads the descriptions and, when one fits your request, asks for it; the harness calls the server, and the answer goes into the model's context window, where the model can use it.

Remote and local servers

A *remote* server runs at its provider, at a web address: FRED and Hugging Face. A *local* server is a program on your computer that the agent starts when it needs it: DuckDB, arXiv and Zotero, run with `uvx`, part of the uv program, which fetches what they need.

The five servers Pollis recommends

Server	What it gives an agent	Runs
FRED	Economic time series from the Federal Reserve Bank of St. Louis: search, notes, observations	remote
Hugging Face	The Hub's models, datasets and Spaces: search, details, files	remote
DuckDB	SQL on CSV, Parquet and JSON files, in an in-memory database	local
arXiv	Papers on arXiv: search, download, read	local
Zotero	Your Zotero library, read only: references, BibTeX, full text	local

Examples

- *Fetch the monthly US unemployment rate, UNRATE, since 2000, and save it as a CSV file.*

FRED finds the series and its observations; the agent writes the file.

- *Which datasets on Hugging Face have daily stock returns?*
- *Load sales.csv and give me the average sale by region.*

DuckDB runs the SQL query on the file, on your computer.

- *Find recent papers on conformal prediction for time series, and summarise their methods.*
- *Give me the BibTeX of Bollerslev (1986) from my Zotero library.*

Setting one up

Compose > MCP Servers shows each server's tools, what it requires and its command, with an **Add** button for each coding agent (**Add and Sign In** for FRED, which asks you to sign in). The **MCP Servers - Recommended** section of the Extensions pane lists the same servers; **Set Up** opens the page at that server.

What leaves your computer

Remote server	Your request to it goes to its provider: your FRED searches go to the St. Louis Fed.
Local server	Its work stays on your computer; arXiv still downloads papers from arXiv.
Cloud model	Whatever a tool returns goes into the context, so it is sent to the model's provider: the full text of a Zotero reference, the rows of a DuckDB query.
Local model	With local servers too, nothing leaves your computer but what they fetch.

> **Add only servers you trust**
>
> A server is a program that acts for your agent: DuckDB can read and write files on your computer. And text an agent fetches, a web page or a paper, may contain instructions written to mislead it. Prefer well-known servers, and read what the agent proposes to do with what it fetched.

See also Agent Skills, page 8. Orchestration, page 12.

Orchestration

Compose > Orchestration

Several models, or several agents, each doing the part it does best.

What orchestration is

So far, one agent has done one task. *Orchestration* arranges several: a cheap model for the small tasks and a strong one for the hard, or two agents that check each other. As a conductor does not play, but decides who plays when, orchestration writes no code; it decides which model or agent does what.

The simplest kind: Model Routing

Not every task needs the strongest model. **Compose > Orchestration > Model Routing** has three *tiers*:

- **Light**: small tasks, on a fast, cheap or local model.
- **Standard**: everyday tasks.
- **Strong**: hard tasks, where a mistake costs most.

Each tier is the command line of an agent, with `{prompt}`. For example:

Light	`ollama launch claude -- {prompt}`, on a local model, free
Standard	`claude --model sonnet {prompt}`
Strong	`claude --model opus {prompt}`

An empty tier runs your main agent, the one chosen in **Compose > Coding Agents**, and so does a tier whose program is not installed. Each task has a tier by default; in **Tasks**, choose another.

Task	Menu	Tier
Polish with Agent	Reports	Light
Write a Vignette, Prepare for Registration	Methods	Standard
Package a Toolbox	New Panels and Toolboxes	Standard
Tables from the Article, Write a Method	Methods	Strong
Create a Panel, Create Panels from a Request	New Panels and Toolboxes	Strong

Why route?

- **Cost**: a small model costs a fraction of a strong one, and a local one nothing.
- **Speed**: small models answer sooner.
- **Privacy**: a local Light tier keeps its tasks on your computer.
- **Quality where it counts**: the tasks whose results must match published values get the strongest model.

Agents that check each other

A model's mistakes are not random: run the same model twice and it may make the same mistake twice. Two *different* models, or one model and a published value, do not share their blind spots. This is the idea behind double data entry, where two people type the same form and every difference is looked at. Three patterns in **Compose > Orchestration** are built on it:

Writer and Checker...	one agent writes; another checks the work.
Two Independent Transcriptions...	two agents copy the same tables from an article, each on its own; where they differ, you decide.
Race...	several agents do the same task, and you compare what they produce.

Not available yet

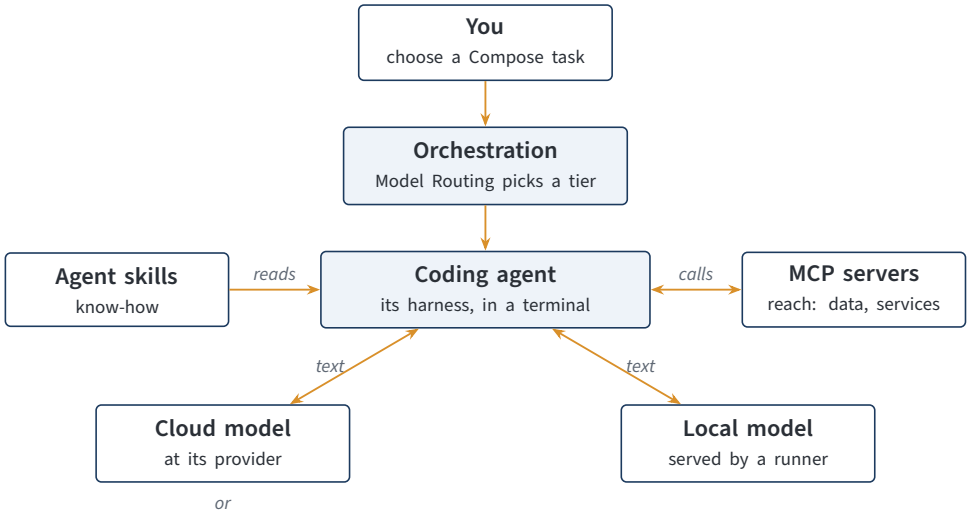
Writer and Checker..., **Two Independent Transcriptions...** and **Race...** are in the **Orchestration** menu but not built yet. Today, Pollis' checks are the tests from published values and the panel validator.

See also Language Models, page 2. Coding Agents and Harnesses, page 6.

The Whole Picture

Compose

How the pieces fit together, and the words to know.



One task, from start to finish

You choose **Compose > Under Construction...** and click **Write Method** beside your method. Model Routing sends the task to the **Strong** tier. Pollis opens a terminal and starts that tier's agent, with a first message naming the write-method skill and the method's test command. The agent reads the skill; if the article is on arXiv, the arXiv server can fetch it; its model writes the code; its harness runs the tests against the published values; and the loop goes on until every test passes. You read the numbers, beside the published ones.

Words to know

Language model (LLM)	A program that continues text, one token at a time; page 2.
Token	A word or part of a word: what models read and write, and what cloud models charge for.
Context window	What a model can read at once: the request, files, the conversation so far.
Knowledge cutoff	The date after which a model knows nothing, unless it is told.
Cloud model	A model run by its provider, reached over the internet with an account.
Open-weight model	A model whose weights are published, so it can run on your computer.
Local model	A model running on your computer: free, private, offline, smaller.
Weights	The trained numbers of a model; their count, in billions, is its size.
Quantised	Kept with fewer bits per weight: a smaller file, slightly less accurate.
Runner	A program that loads a model and serves it on your computer; page 4.
API	The agreed way programs ask and answer; runners speak the APIs of OpenAI and Anthropic.
Coding agent	A model that can act: it reads and writes files and runs commands; page 6.
Harness	The program around the model: loop, tools, permissions, context, memory.
Agent skill	Written know-how for one task, which the agent reads when it needs it; page 8.
MCP server	A server of tools an agent can use, through the Model Context Protocol; page 10.
Orchestration	Arranging several models or agents, each on the part it does best; page 12.
Tier	In Model Routing, Light, Standard or Strong: the agent a kind of task runs on.